

Instituto Tecnológico y de Estudios Superiores de Monterrey.
Campus Querétaro.



**Tecnológico
de Monterrey**

Evidencia Convención de código del equipo.

Construcción de Software y Toma de Decisiones

A01612863 – Luis Enrique De la Cruz Ambrosio

03/06/2026

Explicación

Dentro de la entrega del Avance 3: Diseño de la solución, se trabajó en la creación de la guía de estilo de codificación, la cual sirve para establecer un conjunto de reglas y acuerdos que todo el equipo seguirá al momento de escribir código. El objetivo principal es que aunque cada integrante trabaje en partes distintas del proyecto, el código se vea y se entienda de la misma manera, como si se hubiera escrito por una sola persona.

En esta guía se definen cosas como la forma de nombrar variables, funciones y archivos, el uso correcto de comentarios, el manejo de errores y el formato general que debe tener el código, el tener estas reglas claras desde el inicio ayuda a evitar confusiones y facilita la revisión del trabajo de los compañeros así como también hace que sea mucho más sencillo corregir problemas o agregar nuevas funcionalidades más adelante.

Para construir esta guía se hizo uso del apoyo de inteligencia artificial, la cual ayudó a tomar en cuenta las buenas prácticas que se utilizan comúnmente en el desarrollo de software. De esta forma se logró tener una guía más completa, ordenada y fácil de seguir para todos los miembros del equipo durante el desarrollo del proyecto.

4. Guía de estilo de codificación

4.1 Convención de nombres

Variables

Las variables deberán escribirse en camelCase, iniciando con letra minúscula y utilizando nombres descriptivos.

Ejemplos:

- nombreCliente
- montoOperacion
- estatusAlerta

No se deben usar nombres ambiguos o demasiado cortos como:

- x
- dato
- temp

Funciones

Las funciones deberán escribirse también en camelCase y su nombre debe iniciar con un verbo que describa claramente la acción que realizan.

Ejemplos:

- registrarCliente()
- validarDocumento()
- generarReporte()

Objetos

Los objetos y las estructuras de datos deben usar nombres claros y relacionados con la entidad que a la que representan.

Ejemplos:

- cliente
- operacion
- alerta

Si se llegan a utilizar clases o constructores, estos deberán nombrarse en PascalCase.

Ejemplos:

- Cliente
- Operacion
- Reporte

Archivos

Los archivos deberán tener nombres claros en minúsculas, evitando espacios y caracteres especiales.

Ejemplos:

- clientes.js
- reportes.js
- registroCliente.ejs
- alertas.html

4.2 Uso de strings

Los textos y mensajes dentro del código deben manejarse de forma consistente. Se utilizarán comillas dobles de manera uniforme dentro de cada uno de los archivos, evitando mezclar estilos sin justificación alguna.

Los mensajes mostrados deben ser claros, directos y comprensibles.

Ejemplos:

- "Cliente registrado correctamente"
- "Error al generar el reporte"
- "Documento validado con éxito"

4.3 Funciones

Las funciones deberán cumplir con las siguientes reglas:

1. Tener una sola responsabilidad
2. Ser breves y faciles de entender
3. Evitar una logica muy larga dentro de una sola función
4. Recibir unicamente los parametros necesarios
5. Retornar resultados claros cuando corresponda

Ejemplo:

JavaScript

```
function registrarOperacion(idCliente, tipoOperacion, monto) {  
    // lógica de registro  
}
```

4.4 Parámetros

Los parametros de las funciones deberan de tener nombres descriptivos y estar ordenados. Se deberá evitar funciones con demasiados parametros, ya esto que dificulta su lectura y mantenimiento.

Ejemplo correcto:

JavaScript

```
generarReporte(tipoReporte, fechaInicio, fechaFin)
```

4.5 Comentarios

Los comentarios se utilizarán unicamente cuando estos ayuden a explicar una parte importante del código como lo son procesos complejos.

No se deben de comentar instrucciones obvias.

Ejemplo adecuado:

JavaScript

```
// Se valida que el cliente exista antes de registrar la operación
```

Ejemplo no Adecuado:

JavaScript

```
// Aquí se guarda el dato  
guardarDato();
```

4.6 Objetos y estructura de datos

Los objetos deberán de agrupar información relacionada de forma logica y ordenada. Cada propiedad debe tener un nombre claro y consistente con el resto del sistema.

Ejemplo:

JavaScript

```
const cliente = {  
  idCliente: 1,  
  nombreCliente: "Juan Pérez",  
  rfc: "JUAP000101ABC",  
  correoElectronico: "juan@email.com"  
};
```

4.7 Estructuras de control

Las estructuras de control como lo son if, else, for, while y switch deberán escribirse con llaves, incluso cuando contengan una sola instrucción esto para mejorar la claridad y evitar posibles errores futuros.

Ejemplo:

JavaScript

```
if (clienteExiste) {  
  registrarOperacion();  
} else {  
  mostrarError();  
}
```

Un punto importante que se tomara en cuenta es:

Evitar anidaciones excesivas, mantener condiciones claras y separar bloques complejos en funciones auxiliares

4.8 Indentación y formato

Para poder mantener uniformidad en el proyecto, se utilizarán las siguientes reglas de formato:

1. Indentación de una tabulación
2. Una instrucción por línea
3. Separación visual entre funciones o secciones importantes
4. Líneas con longitud moderada para facilitar la lectura

4.9 Convenciones para los archivos HTML y EJS

En los archivos .html y .ejs se seguirán las siguientes reglas:

1. Usar una indentación consistente de una tabulación.
2. Mantener una estructura ordenada y legible
3. Nombrar clases e identificadores de una forma clara

Ejemplo de nombres claros:

- formCliente
- tablaOperaciones
- mensajeError

4.10 Manejo de errores

El sistema deberá ser capaz de manejar errores de manera clara, mostrando mensajes útiles al usuario y evitando fallos silenciosos. Antes de guardar o procesar información, se deberá validar los datos de entrada.

Ejemplos de validación:

- verificar que los campos obligatorios no estén vacíos
- validar el formato de correo electrónico
- comprobar que el monto sea numérico y positivo

4.11 Consistencia general

Para mantener una consistencia general todo el equipo deberá seguir esta guía durante el desarrollo del proyecto para con esto asegurar que el código mantenga una estructura homogénea. Esto facilitará la colaboración, la revisión del código y futuras mejoras del sistema.

5.5 Quién es responsable de cada actividad

Actividad	Responsable
1.1 Documentación de restricciones adicionales:	Luis Enrique de la Cruz
4. Guía de estilo de codificación	Luis Enrique de la Cruz
3. Bosquejo de Aplicación	Luis Fernando Martinez
1. Modelo Entidad-Relación	Carlos Arturo Gómez Ayala
Revision e integracion del avance	Luis Enrique de la Cruz
2. Tablas correspondientes (Modelo Relacional)	Carlos Arturo Gómez Ayala